

ChargePulse — EV Charging Station Reliability & Fault Prediction SaaS

Complete Architecture & 6-Week MVP Sprint Plan

1. Product Vision

One-liner: ChargePulse predicts EV charger failures before they happen — using custom-built anomaly detection on OCPP telemetry, with zero dependency on any third-party AI.

The problem is massive and validated:

- India has ~29,000+ public charging stations as of mid-2025
- 28-35% are non-functional on any given day (SIAM 2025 survey)
- In some regions, nearly 48% face functional issues (Avaada 2026 report)
- Average downtime per incident: 72 hours
- FAME-III (expected 2026) will likely mandate 95%+ uptime for subsidized stations

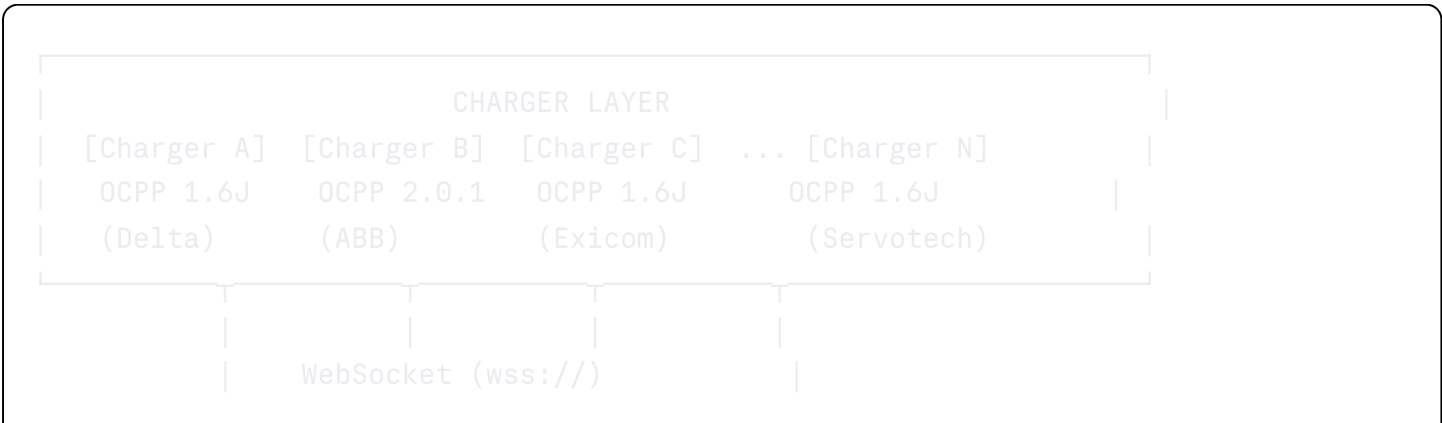
Target customers (India-first):

- Tier-2/Tier-3 CPOs: ChargeZone, Statiq, Zeon, Bolt.Earth, GLIDA, Charzer (50-5,000 stations each)
- Fleet charging operators (Blu-Smart, Lithium Urban, logistics fleets)
- Real estate charging (malls, offices, residential complexes managing 5-50 chargers)

Revenue model: ₹200-500/charger/month SaaS subscription

- 500 chargers = ₹1-2.5L/month
- 5,000 chargers = ₹10-25L/month
- Target Year 1: 2,000 chargers = ₹4-10L/month ARR

2. System Architecture Overview





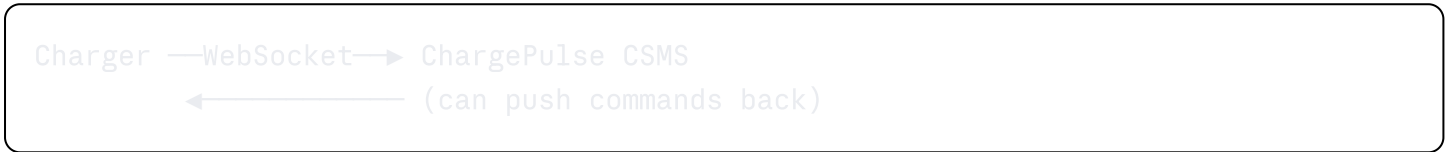




3. OCPP Integration Design (The Core Moat)

3.1 How OCPP Works

OCPP-J uses persistent WebSocket connections. The charger (Charge Point) connects to your server (Central System) and maintains the connection. Communication is bidirectional.



Connection URL pattern:

```
wss://csms.chargepulse.io/ocpp/v16/{charge_point_id}
wss://csms.chargepulse.io/ocpp/v201/{charge_point_id}
```

3.2 Key OCPP Messages to Ingest

From Charger → ChargePulse (the telemetry goldmine):

Message	What It Tells You	Frequency	ML Value
BootNotification	Charger came online, firmware version, vendor, model	On connect	Firmware tracking
StatusNotification	Connector status change (Available, Preparing, Charging, Faulted, Unavailable)	On change	Primary fault signal
MeterValues	Energy (Wh), power (W), voltage (V), current (A), temperature, SoC	Every 30-60s during session	Primary anomaly signal
Heartbeat	"I'm still alive" ping	Every 60-300s	Silent disconnect detection
StartTransaction	Session started, connector ID, meter start	On session start	Session analytics
StopTransaction	Session ended, meter stop, reason (EVDIsconnected, Reboot, PowerLoss, etc.)	On session end	Failure reason signal
DiagnosticsStatusNotification	Diagnostic upload status	On diagnostic	Maintenance tracking
FirmwareStatusNotification	Firmware update progress	On update	Version drift

From ChargePulse → Charger (command capability):

Command	Use Case
RemoteStartTransaction	Test charger responsiveness
Reset	Remote reboot (soft/hard)
ChangeConfiguration	Tune heartbeat interval, meter sampling
GetDiagnostics	Pull charger logs
TriggerMessage	Force a StatusNotification or MeterValues

3.3 OCPP Gateway Implementation

python

```

# gateway/server.py – Core OCPP WebSocket gateway
import asyncio
import logging
import websockets
from datetime import datetime, timezone
from ocpp.routing import on
from ocpp.v16 import ChargePoint as CP16, call_result as result16
from ocpp.v16.enums import (
    RegistrationStatus, Action, ChargePointStatus
)

logger = logging.getLogger("chargepulse.gateway")

class ChargePulseCP(CP16):
    """
    Per-charger handler. One instance per WebSocket connection.
    All OCPP messages route through here.
    """

    def __init__(self, cp_id, connection, org_id, redis_client, db_pool):
        super().__init__(cp_id, connection)
        self.org_id = org_id
        self.redis = redis_client
        self.db = db_pool
        self.connected_at = datetime.now(timezone.utc)
        self.last_heartbeat = self.connected_at

    @on("BootNotification")
    async def on_boot(self, charge_point_vendor, charge_point_model, **kwargs):
        """Charger just booted. Log firmware, vendor, model."""
        await self._publish("boot", {
            "vendor": charge_point_vendor,
            "model": charge_point_model,
            "firmware": kwargs.get("firmware_version"),
            "serial": kwargs.get("charge_point_serial_number"),
        })
        return result16.BootNotificationPayload(
            current_time=datetime.now(timezone.utc).isoformat(),
            interval=60, # Heartbeat every 60s
            status=RegistrationStatus.accepted,
        )

    @on("Heartbeat")
    async def on_heartbeat(self):
        """Track liveness. Missing heartbeats = silent disconnect."""
        self.last_heartbeat = datetime.now(timezone.utc)

```

```

        await self._publish("heartbeat", {})
        return result16.HeartbeatPayload(
            current_time=datetime.now(timezone.utc).isoformat()
        )

@on("StatusNotification")
async def on_status(self, connector_id, error_code, status, **kwargs):
    """
    THE most important signal.
    Status transitions reveal fault patterns.
    """
    await self._publish("status", {
        "connector_id": connector_id,
        "status": status,
        "error_code": error_code,
        "info": kwargs.get("info"),
        "vendor_error_code": kwargs.get("vendor_error_code"),
    })
    return result16.StatusNotificationPayload()

@on("MeterValues")
async def on_meter_values(self, connector_id, meter_value, **kwargs):
    """
    High-frequency telemetry during charging sessions.
    This feeds the anomaly detection pipeline.
    """
    for mv in meter_value:
        samples = {}
        for sv in mv.get("sampled_value", []):
            key = f"{sv.get('measurand', 'Energy.Active.Import.Register')}"
            if sv.get("phase"):
                key += f".{sv['phase']}"
            samples[key] = {
                "value": float(sv["value"]),
                "unit": sv.get("unit"),
                "context": sv.get("context"),
                "location": sv.get("location"),
            }
        await self._publish("meter", {
            "connector_id": connector_id,
            "timestamp": mv.get("timestamp"),
            "samples": samples,
            "transaction_id": kwargs.get("transaction_id"),
        })
    return result16.MeterValuesPayload()

@on("StartTransaction")

```



```

async def on_start_tx(self, connector_id, id_tag, meter_start,
                      timestamp, **kwargs):
    await self._publish("tx_start", {
        "connector_id": connector_id,
        "id_tag": id_tag,
        "meter_start": meter_start,
        "timestamp": timestamp,
    })
    # Return a transaction_id – we generate our own
    tx_id = await self._create_transaction(connector_id, id_tag)
    return result16.StartTransactionPayload(
        transaction_id=tx_id,
        id_tag_info={"status": "Accepted"},
    )

@on("StopTransaction")
async def on_stop_tx(self, meter_stop, timestamp, transaction_id,
                     **kwargs):
    """
    reason field is critical:
    EVDisconnected (normal), Reboot (suspicious), PowerLoss (fault),
    Other (investigate)
    """
    await self._publish("tx_stop", {
        "transaction_id": transaction_id,
        "meter_stop": meter_stop,
        "timestamp": timestamp,
        "reason": kwargs.get("reason", "Unknown"),
    })
    return result16.StopTransactionPayload()

async def _publish(self, event_type: str, payload: dict):
    """Push every event to Redis Stream for downstream processing."""
    msg = {
        "cp_id": self.id,
        "org_id": self.org_id,
        "event": event_type,
        "ts": datetime.now(timezone.utc).isoformat(),
        **{k: str(v) for k, v in payload.items()},
    }
    await self.redis.xadd(
        f"stream:cp:{self.id}",
        msg,
        maxlen=100_000, # Rolling window
    )

async def _create_transaction(self, connector_id, id_tag):

```

```

        """Insert transaction record, return integer ID."""
        # Simplified – real impl uses DB sequence
        return hash(f"{self.id}:{connector_id}:{id_tag}") % 2**31

async def on_connect(websocket, path):
    """
    Handle new charger connections.
    Path format: /ocpp/v16/{charge_point_id}
    """
    cp_id = path.strip("/").split("/")[-1]
    logger.info(f"Charger connected: {cp_id}")

    # Lookup org_id from charger registry (DB call)
    org_id = await lookup_org(cp_id)

    cp = ChargePulseCP(cp_id, websocket, org_id, redis_client, db_pool)

    try:
        await cp.start()
    except websockets.exceptions.ConnectionClosed:
        logger.warning(f"Charger disconnected: {cp_id}")
        await publish_disconnect(cp_id)

async def main():
    server = await websockets.serve(
        on_connect,
        "0.0.0.0",
        9000,
        subprotocols=["ocpp1.6", "ocpp2.0.1"],
        ping_interval=30,
        ping_timeout=10,
    )
    logger.info("ChargePulse OCPP Gateway running on :9000")
    await server.wait_closed()

```

3.4 Vendor-Specific Quirks (The Real Moat)

This is where your competitive advantage compounds over time. Every charger manufacturer implements OCPP slightly differently:

Vendor	Known Quirk	Your Handler
Delta	Sends <code>MeterValues</code> with non-standard <code>measurand</code> strings	Normalize mapping table
ABB	Heartbeat interval sometimes ignored; uses vendor-specific error codes	Custom error code dictionary
Exicom	<code>StatusNotification</code> sometimes missing <code>error_code</code> field	Default to "NoError", flag if status is Faulted
Servotech	Firmware version format inconsistent across models	Regex parser per model
Schneider	Sends extra <code>vendor_id</code> fields not in OCPP spec	Graceful ignore + log

You build a `vendor_profiles/` directory — one YAML config per vendor/model combination. This library grows with every new customer onboarding and becomes your proprietary dataset.

4. Anomaly Detection Pipeline (Your Custom ML Stack)

4.1 Feature Engineering (The Most Important Step)

Every 15 minutes, per charger, you compute a **24-dimensional feature vector** from raw OCPP telemetry:

```
python
```

```
# ml/features.py – Feature extraction from raw OCPP events
```

```
import numpy as np
from dataclasses import dataclass
```

```
@dataclass
```

```
class ChargerFeatureVector:
```

```
    """24 features per 15-min window, per charger."""
```

```
    # === SESSION METRICS (6 features) ===
```

```
    sessions_started: int          # Count of StartTransaction in window
    sessions_completed: int        # Count of StopTransaction (normal)
    sessions_failed: int           # StopTransaction with reason != EVDisconnected
    avg_session_duration_min: float # Mean session length
    avg_energy_delivered_kwh: float # Mean energy per session
    session_completion_rate: float # completed / (completed + failed)
```

```
    # === POWER QUALITY (6 features) ===
```

```
    avg_power_kw: float            # Mean power during active sessions
    std_power_kw: float            # Power stability (high std = problem)
    max_voltage_v: float           # Peak voltage (overvoltage detection)
    min_voltage_v: float           # Min voltage (brownout detection)
    avg_current_a: float           # Mean current draw
    power_factor: float            # Real power / apparent power
```

```
    # === STATUS PATTERNS (6 features) ===
```

```
    status_transitions: int        # Total status changes in window
    time_in_faulted_pct: float     # % of window in Faulted state
    time_in_available_pct: float   # % of window in Available state
    time_in_unavailable_pct: float # % of window in Unavailable state
    error_code_count: int          # Non-"NoError" StatusNotifications
    unique_error_codes: int        # Distinct error types
```

```
    # === CONNECTIVITY (4 features) ===
```

```
    heartbeat_count: int           # Heartbeats received
    heartbeat_gap_max_sec: float    # Longest gap between heartbeats
    heartbeat_gap_std_sec: float    # Variability of heartbeat intervals
    ws_reconnections: int           # WebSocket reconnect events
```

```
    # === TEMPORAL (2 features) ===
```

```
    hour_of_day: float             # Cyclical:  $\sin(2\pi * \text{hour}/24)$ 
    day_of_week: float             # Cyclical:  $\sin(2\pi * \text{day}/7)$ 
```

```
def extract_features(raw_events: list, window_start, window_end):
    """
```

Takes raw OCPP events from Redis/TimescaleDB for one charger in one 15-min window, returns a ChargerFeatureVector.

```
"""
```

```
# ... (implementation extracts from StatusNotification,  
#      MeterValues, Heartbeat, Start/StopTransaction events)  
pass
```

```
def vectorize(fv: ChargerFeatureVector) -> np.ndarray:  
    """Convert to numpy array for model input."""  
    return np.array([  
        fv.sessions_started, fv.sessions_completed, fv.sessions_failed,  
        fv.avg_session_duration_min, fv.avg_energy_delivered_kwh,  
        fv.session_completion_rate,  
        fv.avg_power_kw, fv.std_power_kw, fv.max_voltage_v,  
        fv.min_voltage_v, fv.avg_current_a, fv.power_factor,  
        fv.status_transitions, fv.time_in_faulted_pct,  
        fv.time_in_available_pct, fv.time_in_unavailable_pct,  
        fv.error_code_count, fv.unique_error_codes,  
        fv.heartbeat_count, fv.heartbeat_gap_max_sec,  
        fv.heartbeat_gap_std_sec, fv.ws_reconnections,  
        fv.hour_of_day, fv.day_of_week,  
    ], dtype=np.float32)
```

4.2 Model A: LSTM Autoencoder (Per-Station Anomaly Detection)

This is the core — one small model per charger station, trained on what "normal" looks like for THAT specific station.

python

```

# ml/models/anomaly_detector.py

import torch
import torch.nn as nn

class LSTMAutoencoder(nn.Module):
    """
    Learns to reconstruct "normal" feature sequences.
    High reconstruction error = anomaly.

    Architecture:
    - Encoder: 2-layer LSTM compresses sequence to latent vector
    - Decoder: 2-layer LSTM reconstructs from latent
    - Input: sequence of 24-dim feature vectors (e.g., last 96 windows = 24 hours)
    - If reconstruction error > threshold → anomaly
    """

    def __init__(self, n_features=24, hidden_dim=64, latent_dim=32,
                  n_layers=2):
        super().__init__()
        self.n_features = n_features
        self.hidden_dim = hidden_dim
        self.latent_dim = latent_dim

        # Encoder
        self.encoder_lstm = nn.LSTM(
            input_size=n_features,
            hidden_size=hidden_dim,
            num_layers=n_layers,
            batch_first=True,
            dropout=0.2,
        )
        self.encoder_fc = nn.Linear(hidden_dim, latent_dim)

        # Decoder
        self.decoder_fc = nn.Linear(latent_dim, hidden_dim)
        self.decoder_lstm = nn.LSTM(
            input_size=hidden_dim,
            hidden_size=hidden_dim,
            num_layers=n_layers,
            batch_first=True,
            dropout=0.2,
        )
        self.output_fc = nn.Linear(hidden_dim, n_features)

    def encode(self, x):

```

```

# x shape: (batch, seq_len, n_features)
_, (hidden, _) = self.encoder_lstm(x)
# Use last layer's hidden state
latent = self.encoder_fc(hidden[-1])
return latent

```

```

def decode(self, latent, seq_len):
    # Expand latent to sequence
    hidden = self.decoder_fc(latent)
    # Repeat for each timestep
    decoder_input = hidden.unsqueeze(1).repeat(1, seq_len, 1)
    decoded, _ = self.decoder_lstm(decoder_input)
    output = self.output_fc(decoded)
    return output

```

```

def forward(self, x):
    latent = self.encode(x)
    reconstructed = self.decode(latent, x.size(1))
    return reconstructed

```

```

class AnomalyScorer:

```

```

    """

```

```

    Wraps the autoencoder for production inference.
    Computes per-window anomaly scores.

```

```

    """

```

```

def __init__(self, model: LSTMAutoencoder, threshold: float):
    self.model = model
    self.model.eval()
    self.threshold = threshold

```

```

@torch.no_grad()

```

```

def score(self, feature_sequence: torch.Tensor) -> dict:

```

```

    """

```

```

    Args:

```

```

        feature_sequence: (1, seq_len, 24) tensor

```

```

    Returns:

```

```

        {

```

```

            "anomaly_score": float, # 0.0 = perfectly normal

```

```

            "is_anomaly": bool,

```

```

            "feature_errors": list, # Per-feature reconstruction error

```

```

            "top_anomalous_features": list, # Which features deviate most

```

```

        }

```

```

    """

```

```

    reconstructed = self.model(feature_sequence)

```

```

errors = (feature_sequence - reconstructed).pow(2).squeeze(0)

# Per-window mean squared error
window_errors = errors.mean(dim=1) # (seq_len,)
latest_error = window_errors[-1].item()

# Per-feature error for the latest window
feature_errors = errors[-1].tolist()

# Identify which features are most anomalous
feature_names = [
    "sessions_started", "sessions_completed", "sessions_failed",
    "avg_session_duration", "avg_energy_kwh", "completion_rate",
    "avg_power_kw", "std_power_kw", "max_voltage", "min_voltage",
    "avg_current", "power_factor",
    "status_transitions", "faulted_pct", "available_pct",
    "unavailable_pct", "error_count", "unique_errors",
    "heartbeat_count", "hb_gap_max", "hb_gap_std", "ws_reconnects",
    "hour", "day",
]
ranked = sorted(
    zip(feature_names, feature_errors),
    key=lambda x: x[1], reverse=True
)

return {
    "anomaly_score": latest_error,
    "is_anomaly": latest_error > self.threshold,
    "feature_errors": feature_errors,
    "top_anomalous_features": [
        {"feature": name, "error": err}
        for name, err in ranked[:5]
    ],
}

```

4.3 Model B: Failure-Type Classifier (XGBoost)

When an anomaly is detected, classify WHAT kind of failure is likely:

python


```

# ml/models/failure_classifier.py

import xgboost as xgb
import numpy as np

FAILURE_TYPES = [
    "power_supply",      # Grid brownout, voltage issues
    "connector_fault",   # Physical connector damage, stuck
    "communication_loss", # Network/WebSocket issues
    "payment_system",    # RFID reader, payment gateway
    "firmware_crash",    # Software hang, needs reboot
    "thermal_overload",  # Overheating, derating
    "ground_fault",      # Electrical safety trip
]

class FailureClassifier:
    """
    Given a feature vector + anomaly context, predicts failure type.
    Trained on labeled historical incidents.
    """

    def __init__(self, model_path: str = None):
        self.model = xgb.XGBClassifier(
            n_estimators=200,
            max_depth=6,
            learning_rate=0.1,
            objective="multi:softprob",
            num_class=len(FAILURE_TYPES),
            eval_metric="mlogloss",
            use_label_encoder=False,
        )
        if model_path:
            self.model.load_model(model_path)

    def predict(self, features: np.ndarray) -> dict:
        """
        Returns probability distribution over failure types.
        """
        probs = self.model.predict_proba(features.reshape(1, -1))[0]
        ranked = sorted(
            zip(FAILURE_TYPES, probs),
            key=lambda x: x[1], reverse=True
        )
        return {
            "predicted_failure": ranked[0][0],
            "confidence": float(ranked[0][1]),
        }

```

```
        "all_probabilities": {ft: float(p) for ft, p in ranked},
    }

def train(self, X: np.ndarray, y: np.ndarray):
    """
    X: (n_samples, 24+) – feature vectors + anomaly metadata
    y: (n_samples,) – failure type labels (0-6)
    """
    self.model.fit(
        X, y,
        eval_set=[(X, y)],
        verbose=False,
    )
```

4.4 Training Data Strategy (The Cold-Start Solution)

Phase 1 — Rule-Based Bootstrap (Week 1-4): Before you have enough labeled data for ML, use deterministic rules:

python

```
RULE_BASED_ALERTS = {
    "heartbeat_missing_5min": {
        "condition": lambda fv: fv.heartbeat_gap_max_sec > 300,
        "severity": "high",
        "category": "communication_loss",
    },
    "faulted_state": {
        "condition": lambda fv: fv.time_in_faulted_pct > 0.1,
        "severity": "critical",
        "category": "connector_fault",
    },
    "voltage_anomaly": {
        "condition": lambda fv: fv.min_voltage_v < 200 or fv.max_voltage_v > 260,
        "severity": "high",
        "category": "power_supply",
    },
    "session_failure_spike": {
        "condition": lambda fv: fv.session_completion_rate < 0.7
                        and fv.sessions_started > 3,
        "severity": "medium",
        "category": "connector_fault",
    },
    "power_instability": {
        "condition": lambda fv: fv.std_power_kw > 5.0,
        "severity": "medium",
        "category": "power_supply",
    },
}
```

Phase 2 — Hybrid (Week 4-8): Rules run in parallel with ML anomaly detector. Every rule-triggered alert is auto-labeled and feeds the training pipeline.

Phase 3 — ML-Primary (Week 8+): LSTM autoencoder is the primary detector. Rules become guardrails/fallbacks. XGBoost classifier kicks in with 500+ labeled incidents.

5. Database Schema

```
sql
```

```
-- TimescaleDB hypertables for time-series
```

```
CREATE TABLE ocpp_events (  
    time          TIMESTAMPTZ NOT NULL,  
    cp_id         TEXT        NOT NULL,  
    org_id        UUID        NOT NULL,  
    event_type     TEXT        NOT NULL, -- boot, heartbeat, status, meter, tx_start  
    connector_id  INT,  
    payload       JSONB       NOT NULL,  
    raw_message   JSONB       -- Original OCPP frame for debugging  
);  
SELECT create_hypertable('ocpp_events', 'time');  
CREATE INDEX idx_ocpp_cp ON ocpp_events (cp_id, time DESC);
```

```
CREATE TABLE feature_vectors (  
    time          TIMESTAMPTZ NOT NULL,  
    cp_id         TEXT        NOT NULL,  
    org_id        UUID        NOT NULL,  
    features      FLOAT8[]     NOT NULL, -- 24-dim array  
    anomaly_score FLOAT8,  
    is_anomaly    BOOLEAN  
);  
SELECT create_hypertable('feature_vectors', 'time');
```

```
-- PostgreSQL relational tables
```

```
CREATE TABLE organisations (  
    id            UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
    name          TEXT NOT NULL,  
    plan          TEXT DEFAULT 'starter', -- starter, pro, enterprise  
    created_at    TIMESTAMPTZ DEFAULT now()  
);
```

```
CREATE TABLE chargers (  
    cp_id         TEXT PRIMARY KEY,  
    org_id        UUID REFERENCES organisations(id),  
    vendor        TEXT,  
    model         TEXT,  
    firmware      TEXT,  
    location      GEOGRAPHY(POINT, 4326), -- PostGIS for geo queries  
    address       TEXT,  
    connector_count INT DEFAULT 1,  
    commissioned_at TIMESTAMPTZ,  
    status        TEXT DEFAULT 'unknown',  
    health_score  FLOAT8 DEFAULT 100.0, -- 0-100, decays with anomalies  
    last_seen     TIMESTAMPTZ
```

```

);

CREATE TABLE incidents (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  cp_id       TEXT REFERENCES chargers(cp_id),
  org_id      UUID REFERENCES organisations(id),
  detected_at TIMESTAMPTZ NOT NULL,
  resolved_at TIMESTAMPTZ,
  severity    TEXT NOT NULL,           -- low, medium, high, critical
  failure_type TEXT,                   -- power_supply, connector_fault, etc.
  anomaly_score FLOAT8,
  description TEXT,
  auto_detected BOOLEAN DEFAULT true,
  resolution  TEXT,                   -- Free text: what fixed it?
  -- Resolution feedback feeds back into classifier training
  confirmed_failure_type TEXT          -- Human-labeled ground truth
);

CREATE TABLE alert_configs (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  org_id      UUID REFERENCES organisations(id),
  channel     TEXT NOT NULL,           -- whatsapp, sms, email, webhook, slack
  endpoint    TEXT NOT NULL,           -- phone number, email, URL
  severity_min TEXT DEFAULT 'medium',  -- Only alert on medium+
  active      BOOLEAN DEFAULT true
);

CREATE TABLE ml_models (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  cp_id       TEXT REFERENCES chargers(cp_id),
  model_type  TEXT NOT NULL,           -- anomaly_detector, failure_classifier
  version     INT NOT NULL,
  trained_at  TIMESTAMPTZ DEFAULT now(),
  metrics     JSONB,                   -- {accuracy, f1, threshold, ...}
  model_path  TEXT NOT NULL,           -- S3/local path to .pt or .xgb file
  is_active   BOOLEAN DEFAULT false
);

```

6. API Design

yaml

```
# Core REST API (FastAPI)

# === Authentication ===
POST    /api/v1/auth/login           # Email + password → JWT
POST    /api/v1/auth/register       # New org registration

# === Charger Management ===
GET     /api/v1/chargers             # List all chargers for org
POST    /api/v1/chargers             # Register new charger
GET     /api/v1/chargers/{cp_id}    # Charger details + health score
GET     /api/v1/chargers/{cp_id}/health # Health timeline
GET     /api/v1/chargers/{cp_id}/telemetry # Raw telemetry feed

# === Fleet Dashboard ===
GET     /api/v1/fleet/overview       # Fleet-wide health summary
GET     /api/v1/fleet/map            # All chargers with lat/lng + status
GET     /api/v1/fleet/uptime         # Uptime % over time range

# === Anomalies & Incidents ===
GET     /api/v1/incidents            # List incidents (filterable)
GET     /api/v1/incidents/{id}      # Incident detail
PATCH  /api/v1/incidents/{id}      # Update resolution / confirm type
GET     /api/v1/anomalies/live      # WebSocket: real-time anomaly stream

# === Alerts ===
GET     /api/v1/alerts/config        # List alert configurations
POST    /api/v1/alerts/config        # Create alert channel
PUT     /api/v1/alerts/config/{id}  # Update alert config

# === Analytics ===
GET     /api/v1/analytics/reliability # MTBF, MTTR, uptime by charger/model
GET     /api/v1/analytics/predictions # Upcoming predicted failures
GET     /api/v1/analytics/vendor      # Reliability comparison by vendor/model
```

7. Tech Stack (Complete)

Layer	Technology	Why
OCPP Gateway	Python 3.12 + asyncio + websockets + <code>mobilityhouse/ocpp</code>	Production-grade OCPP lib, async for 10K+ concurrent connections
API Server	FastAPI + uvicorn	Fastest Python REST framework, async, auto-docs

Layer	Technology	Why
Time-Series DB	TimescaleDB (PostgreSQL extension)	Purpose-built for IoT telemetry, compression, continuous aggregates
Relational DB	PostgreSQL 16 + PostGIS	Geo queries for charger map, robust JSONB for flexible payloads
Message Queue	Redis 7 Streams	Lightweight, fast, consumer groups for parallel processing
ML Training	PyTorch 2.x (LSTM autoencoder)	Full control, custom architectures, no pre-trained model deps
ML Classification	XGBoost / LightGBM + scikit-learn	Best-in-class for tabular data, no GPU needed
ML Serving	In-process Python (no separate serving infra for MVP)	Keep it simple — model loads in the feature engine process
Frontend	React 18 + Tailwind + Recharts + Leaflet	Fast dashboard development, map visualization
Deployment	Docker Compose → single VPS (Hetzner/DigitalOcean)	₹3-5K/month for MVP scale (1000 chargers)
Alerts	MSG91 (SMS, India), Twilio (WhatsApp), AWS SES (email)	India-first: MSG91 is cheapest for Indian SMS

8. Six-Week MVP Sprint Plan

Week 1: Foundation

- ☐ Set up monorepo: `chargepulse/gateway`, `chargepulse/api`, `chargepulse/ml`, `chargepulse/web`
- ☐ Docker Compose: PostgreSQL + TimescaleDB + Redis
- ☐ OCPP gateway: handle BootNotification, Heartbeat, StatusNotification
- ☐ Store all messages in TimescaleDB
- ☐ Test with `ocppsim` (OCPP charge point simulator)

Deliverable: Gateway accepts simulated charger connections, stores events

Week 2: Full OCPP + Feature Engine

- ☐ Complete OCPP 1.6 message handlers (MeterValues, Start/StopTransaction)
- ☐ Redis Streams pipeline: events → feature engine consumer

- ☐ Feature extraction: compute 24-dim vector per 15-min window
- ☐ Store feature vectors in TimescaleDB
- ☐ Heartbeat watchdog: detect missing heartbeats, flag disconnects

Deliverable: Feature vectors flowing into DB from simulated charger traffic

Week 3: Rule-Based Alerts + API

- ☐ Implement rule-based alert engine (5 core rules from Section 4.4)
- ☐ FastAPI: auth (JWT), charger CRUD, incident listing
- ☐ Alert dispatch: email (SES) + SMS (MSG91) + webhook
- ☐ Multi-tenant org isolation

Deliverable: Rules detect simulated faults, send alerts via email/SMS

Week 4: ML Pipeline + Dashboard Shell

- ☐ LSTM Autoencoder: define architecture, training loop
- ☐ Generate synthetic "normal" data from simulator (2 weeks of data)
- ☐ Train first per-station model, compute anomaly scores
- ☐ React dashboard: login, charger list, real-time status table
- ☐ Health score computation (rolling 7-day weighted anomaly score)

Deliverable: ML model running on simulated data, basic dashboard shows charger health

Week 5: Dashboard + Real Charger Integration

- ☐ Dashboard: fleet map (Leaflet), anomaly timeline, incident detail view
- ☐ WebSocket live feed: real-time status updates on dashboard
- ☐ Onboard 1 real CPO pilot (target: ChargeZone or Statiq or local fleet)
- ☐ Adapt vendor profile for pilot's charger hardware
- ☐ Start collecting real data

Deliverable: Dashboard showing real charger data from pilot partner

Week 6: Polish + Launch Prep

- ☐ Dashboard: uptime analytics, vendor comparison, prediction panel
- ☐ XGBoost failure classifier (trained on rule-labeled + manual incidents)
- ☐ Alert configuration UI in dashboard
- ☐ Landing page + documentation
- ☐ Pricing page: ₹200/charger/month starter, ₹400/charger/month pro
- ☐ Deploy to production (Hetzner CPX31: 4 vCPU, 8GB RAM, ₹~3K/month)

Deliverable: Production-ready MVP, pilot CPO live, landing page up

9. Go-to-Market (India-First)

Pilot Strategy

- 1. **Target 3 CPOs in Chennai/Coimbatore** with 50-500 chargers each
- 2. **Offer:** Free 30-day pilot, you deploy ChargePulse as a CSMS proxy (their chargers point to your gateway, you forward to their existing backend)
- 3. **Prove:** Show them incidents you caught that their existing system missed
- 4. **Convert:** ₹200/charger/month after pilot

Sales Messaging

"Your chargers are down 28% of the time. You only find out when a driver complains. ChargePulse tells you 4 hours BEFORE a charger fails — and tells you exactly what's wrong. ₹200/charger/month. FAME-III will mandate 95% uptime. Get ahead of it."

Competitive Positioning

	ChargePulse	ChargerHelp (US)	Existing CSMS (SteVe etc.)
Predictive alerts	✔ Custom ML	⚠ Services-led	✗ None
India-ready	✔ Built for Indian CPOs	✗ US-focused	⚠ Generic
Self-hosted AI	✔ Zero API dependency	✗ Proprietary cloud	N/A
Price	₹200-500/charger/mo	\$\$\$\$ enterprise	Free (no analytics)
Vendor-agnostic	✔ Any OCPP charger	⚠ Limited vendors	✔ OCPP

10. Revenue Projections

Milestone	Chargers	MRR (₹)	Timeline
Pilot	100	₹0 (free trial)	Month 1-2
First paying	200	₹40K-1L	Month 3
Traction	1,000	₹2-5L	Month 6
Growth	5,000	₹10-25L	Month 12
Scale	20,000	₹40L-1Cr	Month 24

Unit economics:

- Server cost per 1,000 chargers: ~₹8K/month (Hetzner)
 - Gross margin: ~90%+
 - CAC: near-zero if pilot-driven (CPOs have a pain point, you solve it live)
-

11. Expansion Roadmap (Post-MVP)

1. **OCPP 2.0.1 full support** — ISO 15118, Device Model, improved security profiles
2. **Automated remediation** — Send `Reset` command when firmware crash detected
3. **Fleet optimization** — Recommend charger placement based on utilization data
4. **Insurance integration** — Provide uptime certificates for insurance premium discounts
5. **Battery swap stations** — Extend to Sun Mobility / Battery Smart infrastructure
6. **Southeast Asia expansion** — Thailand, Indonesia, Vietnam EV charging is 2-3 years behind India
7. **FAME-III compliance module** — Auto-generate uptime reports for subsidy claims